INTERNATIONAL CYBERSECURITY AND DIGITAL FORENSICS ACADEMY

CORE102

LINUX FIREWALL AND SECURE REMOTE ACCESS

Laboratory 4 — Practical Evidence Report

Field	Submission detail
Student	CLEMENT ENEJI
Registration Number	_____
Course	CORE102 TCP/IP Networking and Linux Network Tools
Instructor	Aminu Idris AMCPN
Laboratory	Laboratory 4 — Linux Firewall and Secure Remote Access
Environment	Kali Linux / Ubuntu client on authorised isolated lab network
Submission Date	30 September 2026

Maximum assessment value: 100 marks

Contents

- 1. Executive Summary
- 2. Laboratory Context and Topology
- 3. Assessment Coverage Matrix
- 4. Part 1 — Baseline and Recovery Preparation
- 5. Part 2 — Controlled Test Service
- 6. Part 3 — UFW Policy Implementation
- 7. Part 4 — Firewall Rule Verification
- 8. Part 5 — Safe iptables Custom-Chain Exercise
- 9. Part 6 — SSH Key Authentication
- 10. Part 7 — SSH Server Hardening
- 11. Part 8 — Final Exposure Review and Cleanup
- 12. Reflection Questions
- 13. Evidence Integrity and Residual Risk Notes
- 14. Final Submission Checklist

1. Executive Summary

This report documents the practical work completed for ICDFA CORE102 Laboratory 4, focused on Linux host firewall controls and secure remote administration. The practical sequence covered a temporary HTTP test service, UFW policy enforcement, client-side connectivity tests, inspection of UFW/iptables counters and logs, a non-attached iptables custom-chain exercise, Ed25519 SSH key generation and deployment, SSH server hardening, and final cleanup.

The supplied evidence demonstrates successful SSH connectivity, successful access to the temporary HTTP service when permitted, blocking of TCP/23, creation and complete removal of the CORE102_LAB custom chain, Ed25519 key deployment, successful public-key login, successful sshd configuration validation/reload, and final removal of the temporary HTTP service.

Important evidence qualification: Two presentation inconsistencies are preserved and explicitly identified rather than silently corrected: early controlled-service screenshots use 192.168.56.101/192.168.56.102, while later firewall evidence uses 192.168.56.10/192.168.56.20; and one draft table entry describes an HTTP/8080 result as HTTP 200 while later evidence shows the port timing out after the rule/service state changed.

2. Laboratory Context and Topology

The laboratory scenario requires hardening a Linux server before it joins an internal training network. The workbook specifies a server VM and an authorised client VM on an isolated 192.168.56.0/24 network, with SSH permitted before a default-deny incoming policy is enabled.

System	Address in workbook	Role
Server VM	192.168.56.10	UFW, service inventory and SSH hardening
Client VM	192.168.56.20	Authorised connectivity and firewall testing
Private network	192.168.56.0/24	Host-only/internal virtual network

3. Assessment Coverage Matrix

This matrix maps the supplied work to the assessment areas in the laboratory workbook. It is a coverage map, not a claim of marks awarded.

Assessment area	Marks	Evidence included
Baseline and service inventory	15	Baseline service table; evidence limitations identified
UFW policy	20	UFW default-policy and numbered-rule/cleanup evidence
Firewall testing	15	SSH/8080/Telnet tests; UFW logs and counters
iptables chain exercise	10	Rule anatomy, zero-reference explanation and removal proof
SSH key authentication	20	Ed25519 generation, fingerprint, installation and successful login
SSH hardening and recovery	15	Backup, drop-in, sshd -t, reload and retest
Final review and report	5	Cleanup and final exposure evidence
TOTAL	100	Complete practical evidence package as supplied

4. Part 1 — Baseline and Recovery Preparation

The completed answer records two listening services from the baseline inventory: SSH on TCP/22 and a NetworkManager-managed UDP/546 listener bound to the link-local IPv6 address on eth0.

Port / protocol	Bind address	Process	Expected purpose	Disposition
22/TCP	0.0.0.0 / [::]	sshd (PID 15066)	Remote SSH administrative access	Keep
546/UDP	[fe80::a00:27ff:fe3c:3f0f]%eth0	NetworkManager (PID 594)	DHCPv6 client auto-configuration	Keep

Evidence limitation: The supplied answer contains the populated service-inventory table, but not a complete screenshot set for every baseline command listed in the workbook. Those outputs are not presented as if captured.

5. Part 2 — Controlled Test Service

A temporary Python HTTP server was used to create a controlled TCP/8080 test target. The evidence shows the service listening on 0.0.0.0:8080 and serving the CORE102 test content.

```
mkdir -p ~/core102_lab4/web
echo 'CORE102 authorised firewall test' &gt; ~/core102_lab4/web/index.html
python3 -m http.server 8080 --bind 0.0.0.0
```

Evidence 1 — Temporary HTTP service listening on TCP/8080

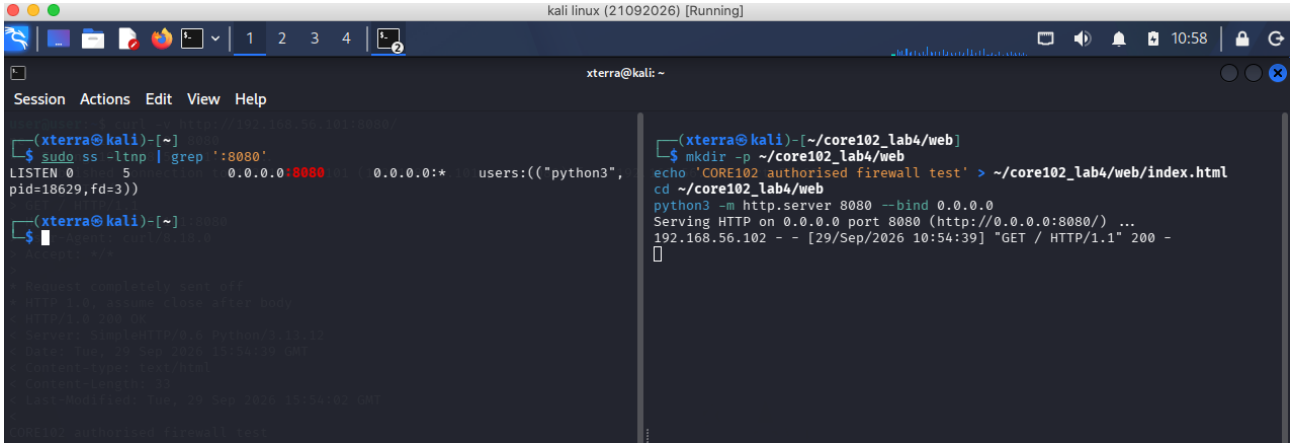


Figure 1. Temporary HTTP service listening on TCP/8080

Evidence 2 — Client-side pre-firewall HTTP/8080 connectivity test

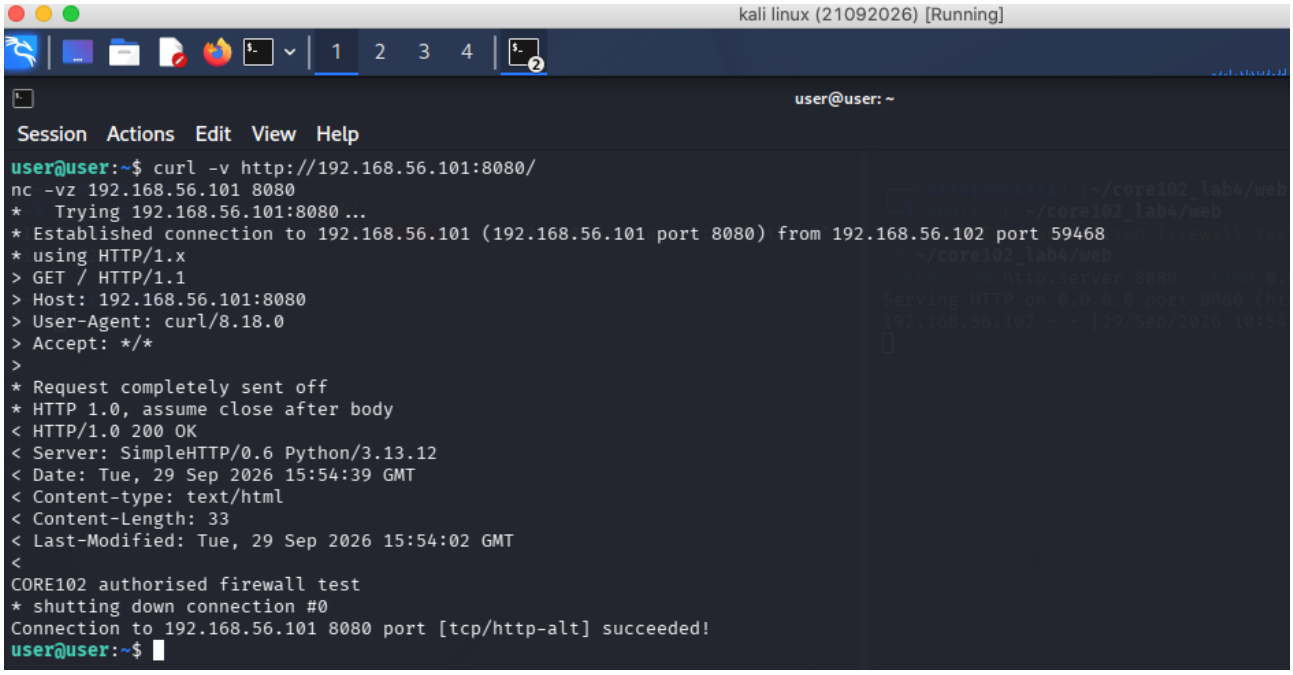


Figure 2. Client-side pre-firewall HTTP/8080 connectivity test

Evidence note: This early connectivity evidence uses 192.168.56.101/192.168.56.102, whereas later firewall evidence uses 192.168.56.10/192.168.56.20.

6. Part 3 — UFW Policy Implementation

The firewall was enabled with an incoming default-deny policy and outgoing default-allow policy. SSH/TCP 22 was explicitly allowed before firewall activation. The captured implementation evidence shows SSH allowed from Anywhere rather than restricted to the 192.168.56.0/24 subnet specified in the workbook.

```
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow 22/tcp
sudo ufw enable
sudo ufw status verbose
```

Evidence 3 — UFW enabled with default-deny incoming and SSH permitted

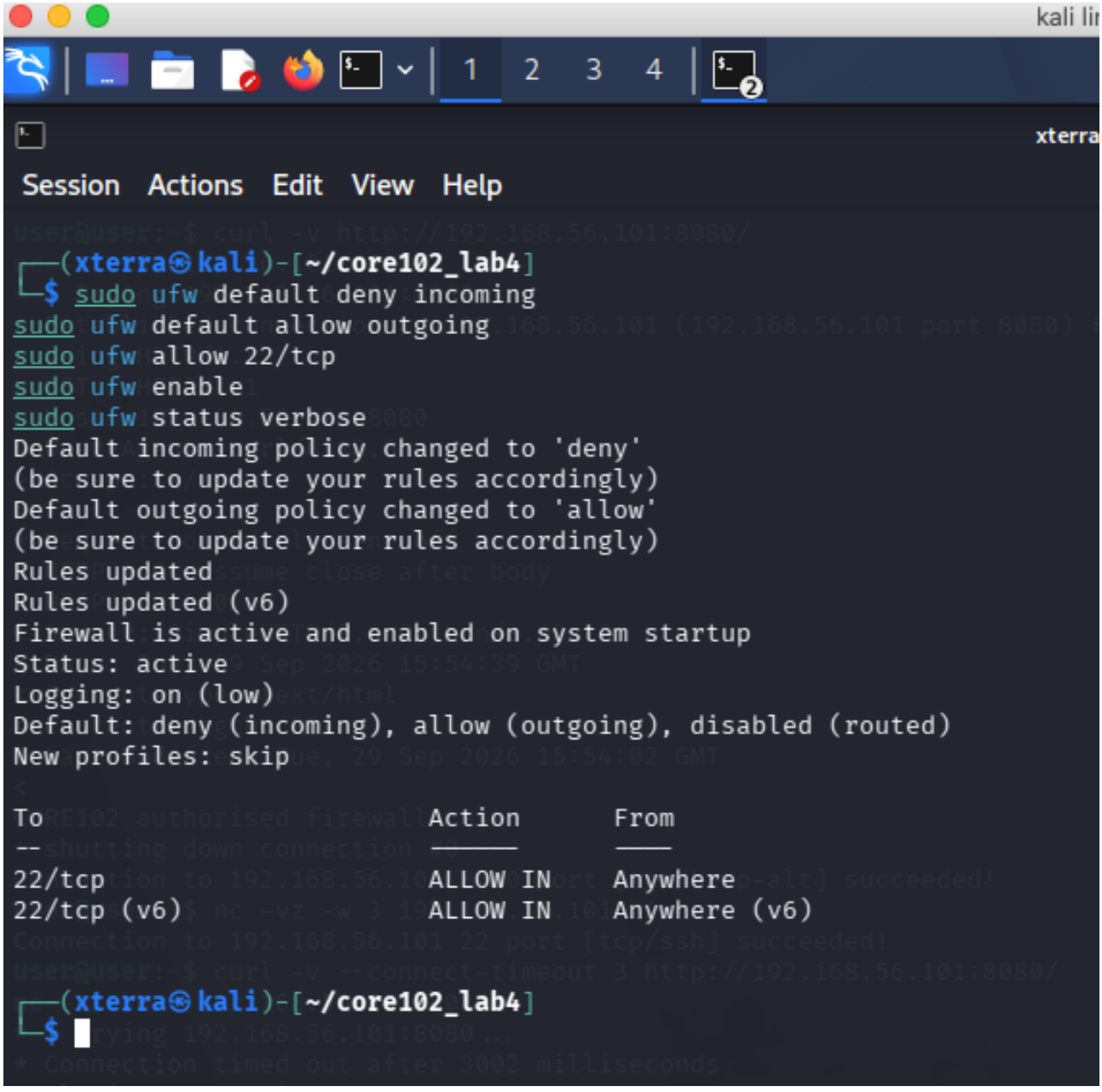


Figure 3. UFW enabled with default-deny incoming and SSH permitted

Configuration-scope note: The workbook's intended command is a subnet-scoped SSH rule. The supplied screenshot instead shows 22/tcp ALLOW IN Anywhere. This report records the actual evidence and does not describe the rule as subnet-restricted.

7. Part 4 — Firewall Rule Verification

The client-side tests demonstrate SSH/22 reachability, HTTP/8080 reachability while its allow rule is present, and TCP/23 blocking. Server-side evidence also shows UFW block logging and iptables counters.

Test	Expected	Evidence-supported observation	Interpretation
SSH from authorised client	Allowed	192.168.56.10:22 succeeded	SSH permitted
HTTP/8080 while allowed	Allowed	192.168.56.10:8080 succeeded; HTTP response returned	Temporary web service reachable
TCP/23 Telnet	Denied	Connection timed out; UFW BLOCK entries recorded	Traffic dropped
HTTP/8080 after cleanup	Denied/no listener	Final test reports timeout and unreachable	Service/rule removed

Evidence 4 — Client tests: allowed SSH/8080 followed by blocked 8080 after state change

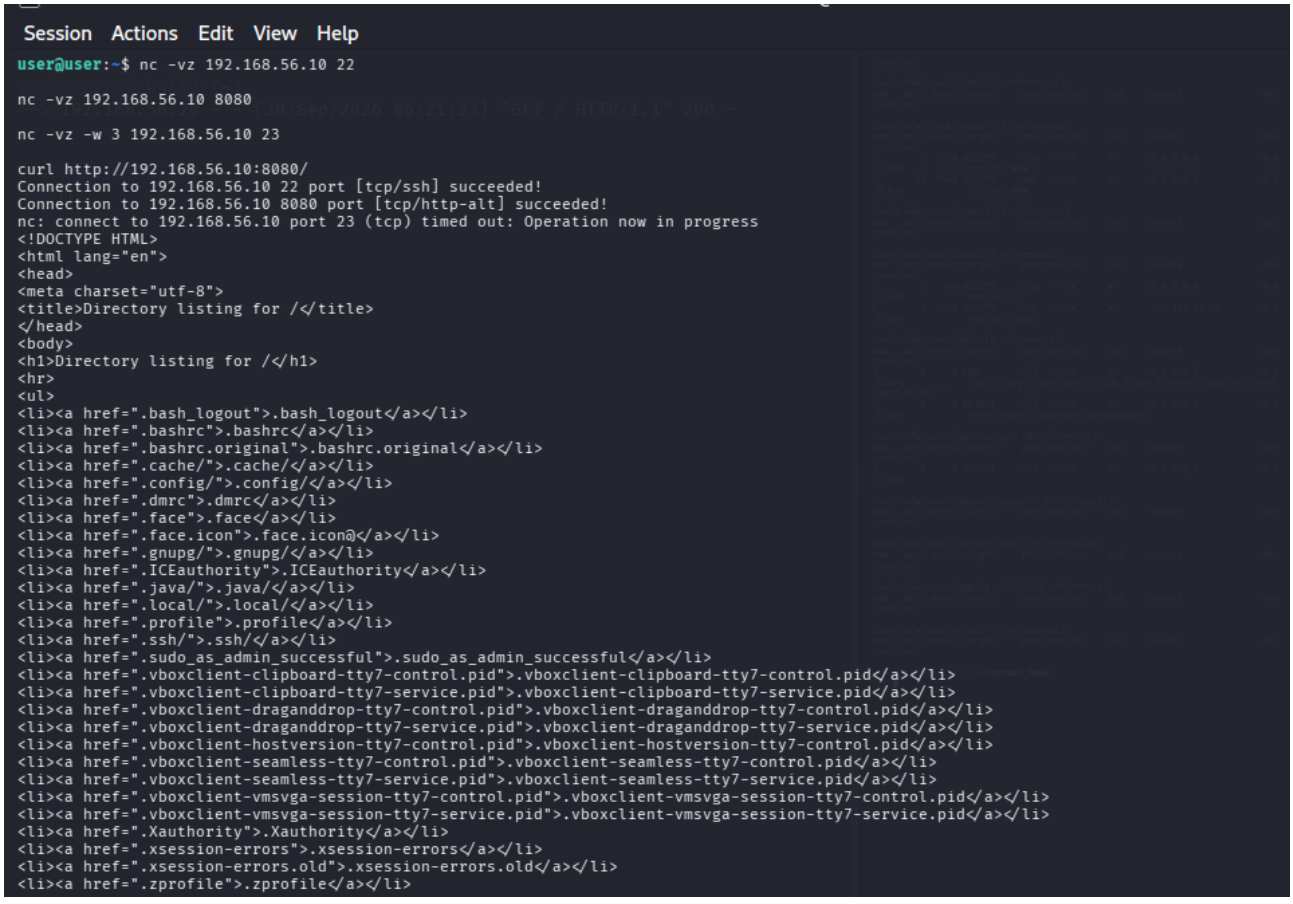


Figure 4. Client tests: allowed SSH/8080 followed by blocked 8080 after state change

Evidence 5 — UFW numbered rules, kernel block logs and iptables counters

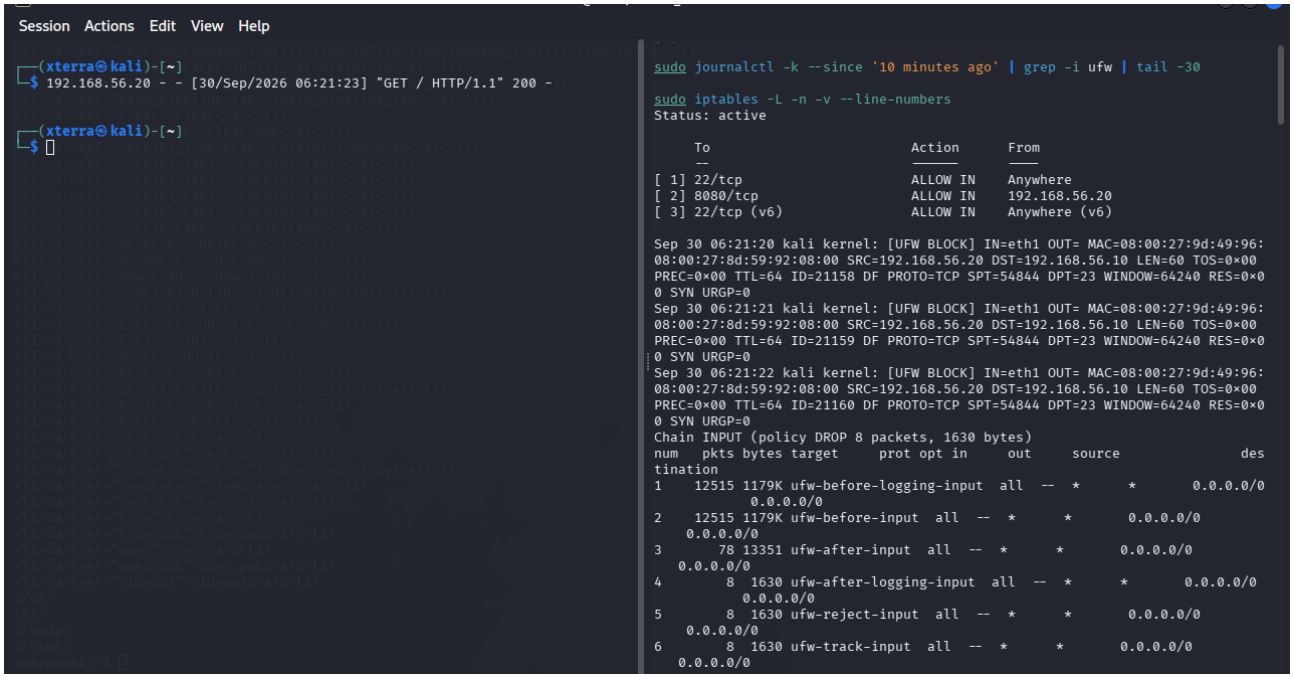


Figure 5. UFW numbered rules, kernel block logs and iptables counters

Evidence 6 — Client-side SSH, HTTP/8080 and Telnet tests

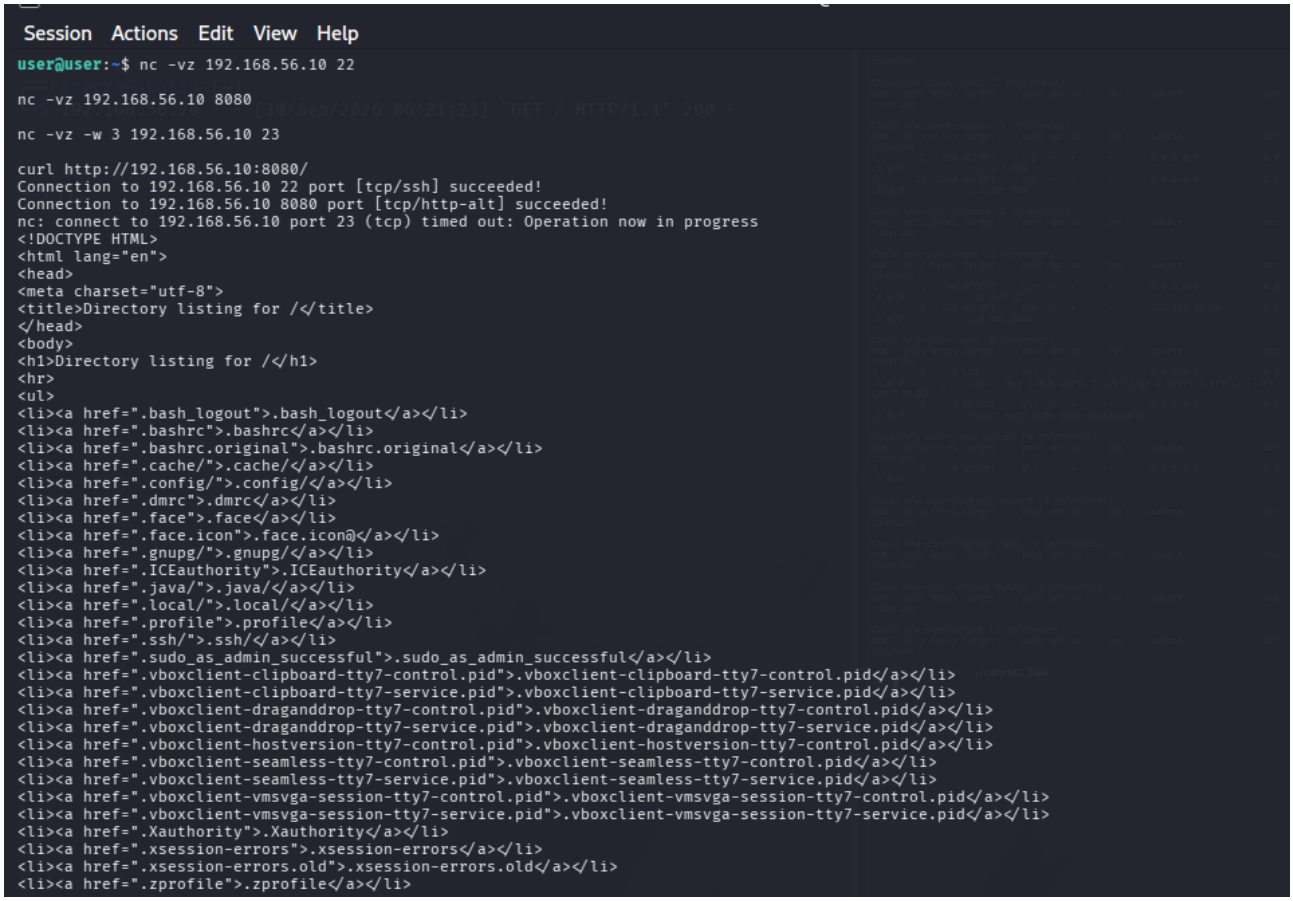


Figure 6. Client-side SSH, HTTP/8080 and Telnet tests

Evidence reconciliation: The draft answer table contains an 'HTTP 8080 from other client' row with an HTTP 200 observation, which conflicts with the intended denied result. The screenshots show successful access during the allowed state and a timeout after the rule/service state changed; the report does not claim a denied HTTP 200 result.

8. Part 5 — Safe iptables Custom-Chain Exercise

A user-defined iptables chain named CORE102_LAB was created, populated with two rules, inspected, flushed and deleted. The chain remained unreferenced, so it did not alter live INPUT, OUTPUT or FORWARD traffic.

Rule element	TCP/23 rule	TCP/22 rule
Tool	iptables	iptables
Action	-A CORE102_LAB	-A CORE102_LAB
Protocol match	-p tcp	-p tcp
Destination port	--dport 23	--dport 22
Target	DROP	ACCEPT

```
sudo iptables -N CORE102_LAB
sudo iptables -A CORE102_LAB -p tcp --dport 23 -j DROP
sudo iptables -A CORE102_LAB -p tcp --dport 22 -j ACCEPT
sudo iptables -L CORE102_LAB -n -v --line-numbers
sudo iptables -S CORE102_LAB
sudo iptables -F CORE102_LAB
sudo iptables -X CORE102_LAB
```

Evidence 7 — Custom-chain creation, inspection and complete removal

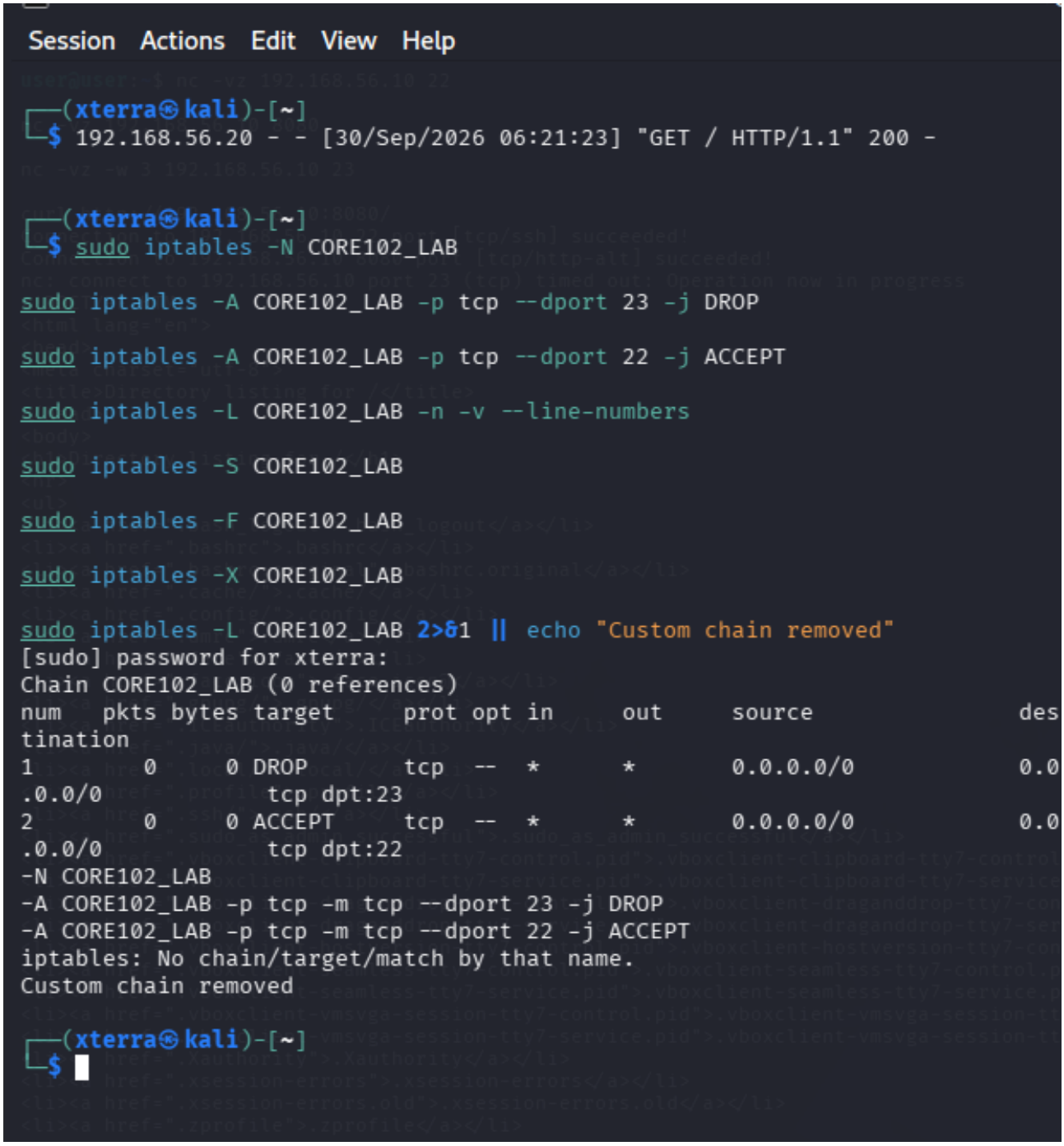


Figure 7. Custom-chain creation, inspection and complete removal

A user-defined chain is not automatically traversed by packets entering the firewall. It must be reached by a jump from a built-in chain such as INPUT. The screenshot shows '0 references' before removal, confirming that CORE102_LAB was not attached to the live traffic path.

Cleanup result: After flushing and deleting the chain, the verification command returned 'Custom chain removed'.

9. Part 6 — SSH Key Authentication

An Ed25519 key pair was generated on the client using a dedicated filename and the laboratory label. The public-key fingerprint was displayed, the public key was installed on the server with ssh-copy-id, and a subsequent SSH session successfully authenticated with the key.

```
ssh-keygen -t ed25519 -a 64 -f ~/.ssh/core102_ed25519 -C "CORE102 authorised lab"
ssh-keygen -lf ~/.ssh/core102_ed25519.pub
ssh-copy-id -i ~/.ssh/core102_ed25519.pub xterra@192.168.56.10
ssh -i ~/.ssh/core102_ed25519 xterra@192.168.56.10
```

Item	Evidence-supported value
Key type	ED25519
Key filename	~/.ssh/core102_ed25519
Fingerprint	SHA256:KuKWlxkYnHEwgtsItRXRksCghoNOsnMtCyh3/fl0gV0
Server	192.168.56.10
Successful authentication	Yes — accepted publickey/session opened

Evidence 8 — Ed25519 key generation, fingerprint and public-key installation

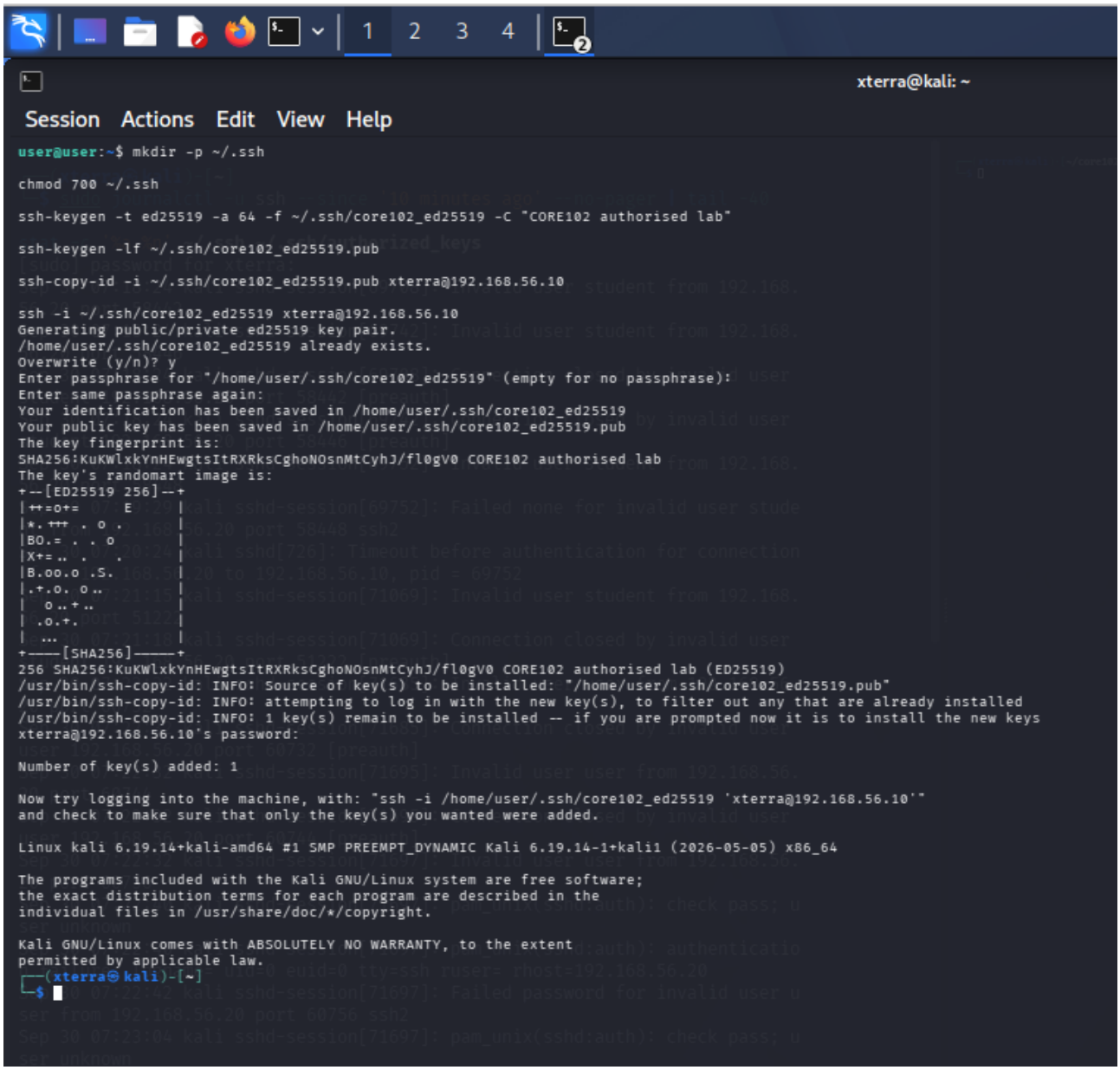


Figure 8. Ed25519 key generation, fingerprint and public-key installation

Evidence 9 — Successful key-based SSH login and sshd public-key acceptance

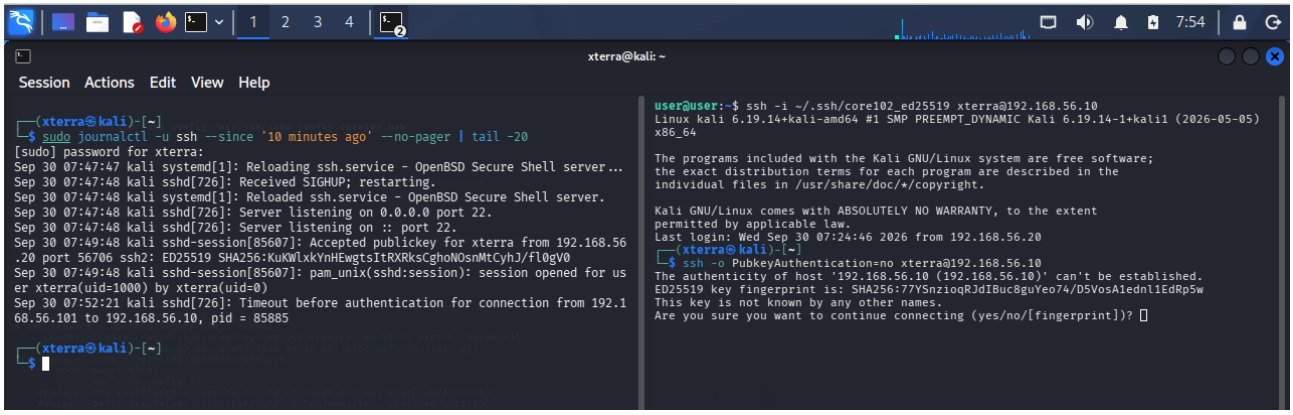


Figure 9. Successful key-based SSH login and sshd public-key acceptance

Private-key handling: Only the public-key fingerprint and setup process are documented. Private-key contents are intentionally not reproduced.

10. Part 7 — SSH Server Hardening

The SSH server configuration was backed up, a dedicated drop-in configuration file was created, the configuration was validated with sshd -t, and the SSH service was reloaded. The evidence then shows a successful public-key login.

```
PermitRootLogin no
PubkeyAuthentication yes
PasswordAuthentication no
KbdInteractiveAuthentication no
```

Setting	Value	Purpose
PermitRootLogin	no	Prevents direct root SSH login
PubkeyAuthentication	yes	Enables public-key authentication
PasswordAuthentication	no	Disables password-based authentication after key verification
KbdInteractiveAuthentication	no	Disables keyboard-interactive authentication

Evidence 10 — SSH configuration backup, hardening drop-in, validation and reload

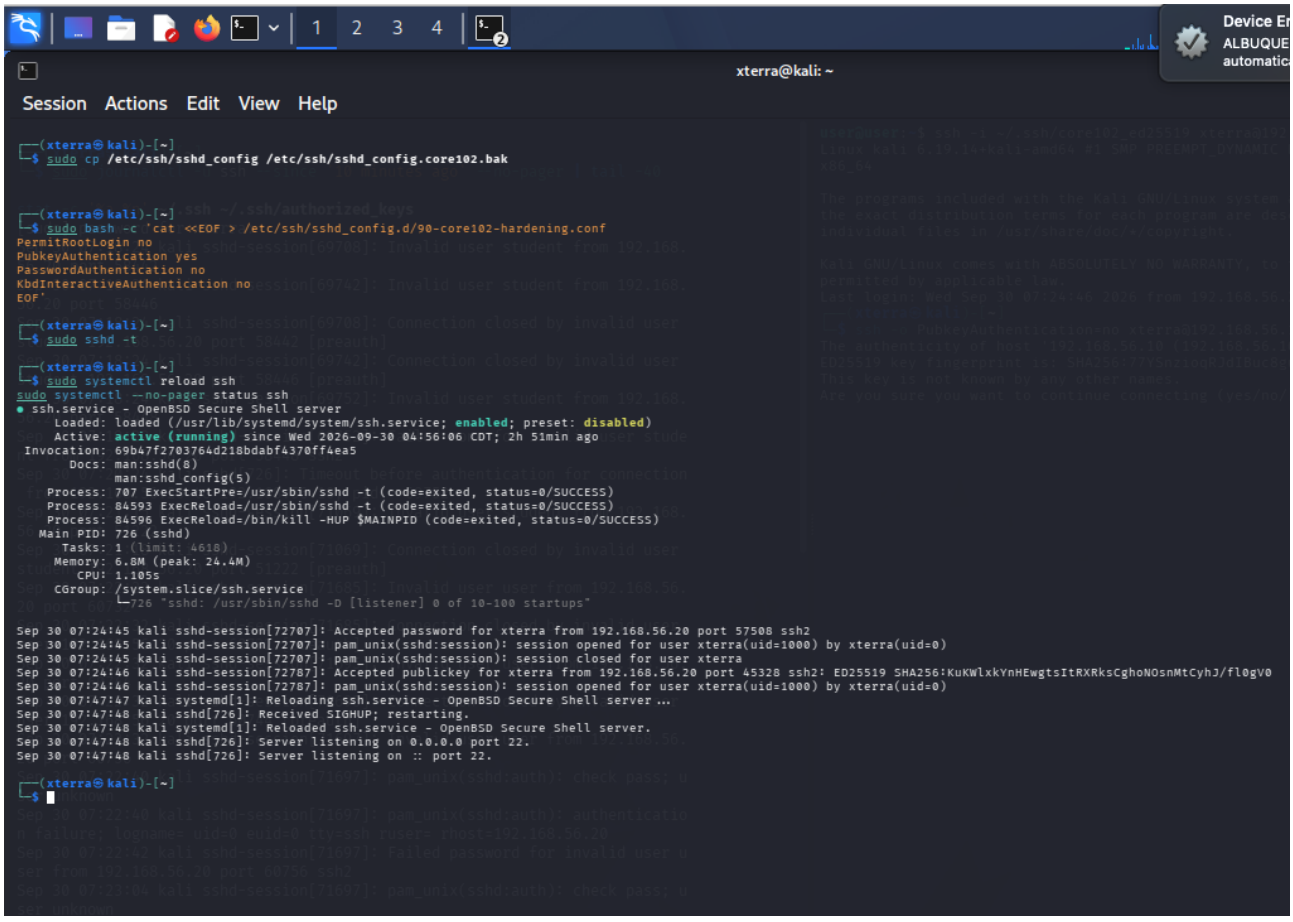


Figure 10. SSH configuration backup, hardening drop-in, validation and reload

Evidence 11 — Second-session key-based login and sshd confirmation

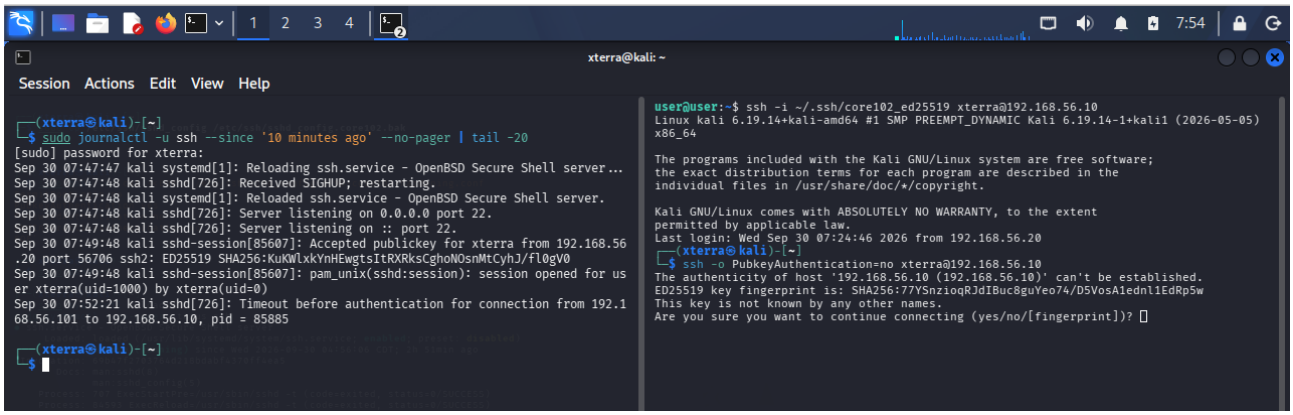


Figure 11. Second-session key-based login and sshd confirmation

Hardening gate: The supplied evidence shows successful public-key authentication and successful sshd configuration validation before the reload was relied upon.

11. Part 8 — Final Exposure Review and Cleanup

The temporary 8080 UFW rule was deleted, the rule list was rechecked, and ss confirmed that the temporary HTTP service was no longer listening. The final socket inventory shown in the evidence contains SSH on TCP/22 as the remaining TCP listener.

```
sudo ufw status numbered
sudo ufw delete 2
sudo ufw status numbered
sudo ss -ltnp | grep ':8080' || echo "Temporary web service stopped"
sudo ss -tulpn
```

Evidence 12 — Deletion of the temporary 8080 rule and final socket inventory

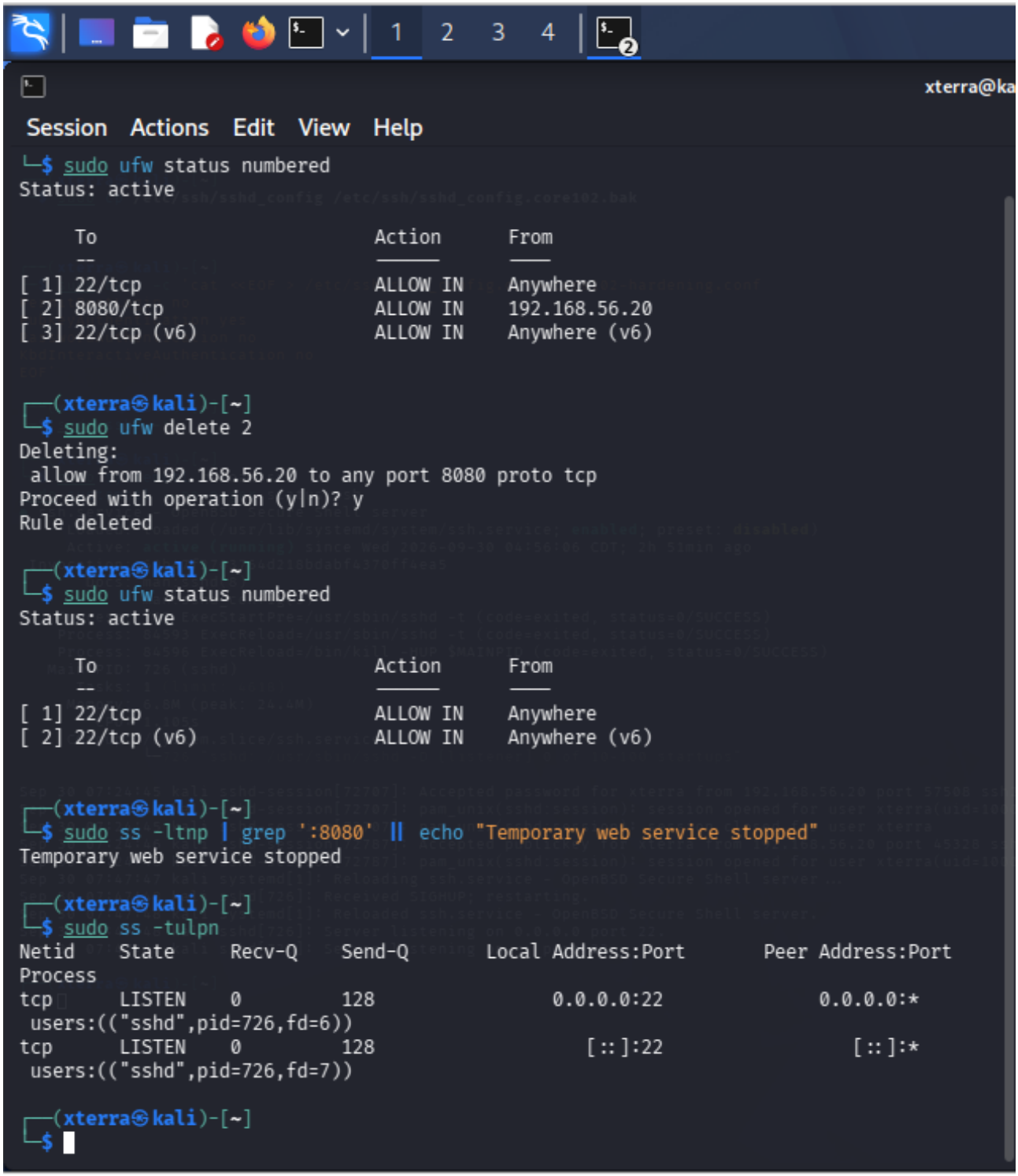


Figure 12. Deletion of the temporary 8080 rule and final socket inventory

Evidence 13 — Final client verification: SSH key login active and port 8080 unreachable

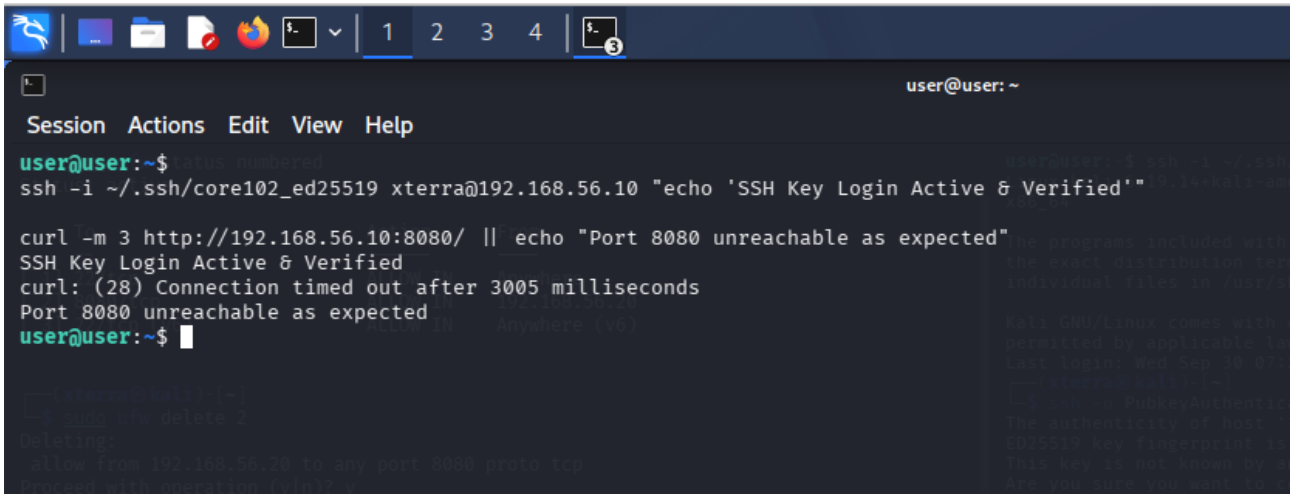


Figure 13. Final client verification: SSH key login active and port 8080 unreachable

Cleanup result: The final evidence explicitly reports 'Temporary web service stopped' and shows no TCP/8080 listener. The client confirms SSH key login remains active while port 8080 is unreachable.

12. Reflection Questions

1. Why must SSH access be allowed before a default-deny firewall is enabled?

A default-deny incoming policy drops traffic that has not been explicitly permitted. If SSH is the management path and port 22 is not allowed first, enabling the policy can terminate or prevent administrative access. The laboratory therefore requires SSH to be permitted and tested before enabling the deny policy.

2. What is the difference between an open port and an authorised service?

An open/listening port means a process is accepting traffic on that port. Authorisation is a policy decision about whether that service and its exposure are permitted for the intended hosts and purpose. A listening service can therefore be technically reachable but still outside the approved security policy.

3. Why does the server need the public key while the client must protect the private key?

The server uses the public key to verify a signature produced with the corresponding private key. The private key is the secret credential; disclosure would allow another party to attempt authentication as the key holder. The public key is designed to be installed on the server and does not reveal the private key.

4. Which rollback step would recover access after an invalid SSH configuration?

Use the VM console rather than relying on the broken SSH path, restore the backed-up /etc/ssh/sshd_config (or remove the faulty drop-in), run sudo sshd -t to validate the restored configuration, and reload the SSH service only after validation succeeds.

13. Evidence Integrity and Residual Risk Notes

The following points are retained so that the report accurately represents the supplied practical evidence and does not overstate what was demonstrated:

• SSH scope: the captured UFW rule permits TCP/22 from Anywhere, while the workbook specifies the 192.168.56.0/24 laboratory subnet.

• Addressing stages: early HTTP evidence uses .101/.102; later firewall evidence uses .10/.20.

• HTTP/8080 state: screenshots show successful access while allowed and timeout after cleanup.

• Baseline completeness: the supplied answer lacks the full before-change interface/route/service/firewall screenshot set.

• Temporary web-service exposure: one client response shows a directory listing during the temporary service test; the service was subsequently stopped.

• SSH authentication: earlier failed/invalid-user attempts are visible in logs, followed later by accepted ED25519 public-key authentication.

Residual-risk interpretation: The main residual configuration concern visible in the supplied evidence is the broader-than-specified SSH source scope (Anywhere). If strict workbook scope compliance is required, capture a corrected subnet-scoped rule before final submission.

14. Final Submission Checklist

- ■ Student name and registration number completed.
- ■ Report reviewed for screenshots, captions and command/output visibility.
- ■ No private SSH key, password, token or secret material included.
- ■ Final filenames use the registration-number convention.
- ■ Temporary HTTP service stopped and TCP/8080 rule removed.
- ■ Custom iptables chain removed.
- ■ SSH key-based access verified in a separate session.
- ■ SSH configuration validated with sshd -t before reload.
- ■ Any required correction to SSH source scope is evidenced before final upload.

Recommended filenames:
REGNO_CORE102_Lab4_Report.pdf
REGNO_CORE102_Lab4_Files.zip